

CEN4025C Module 3 Project

This project extends the existing TaskManagerFrontEnd by integrating it with the backend APIs using Angular's **HttpClient**. The frontend will fetch users, tasks, and subtasks using RESTful services.

API Endpoints Available

The Java controllers in the provided TaskManager Spring application provide the following endpoints:

Tasks API (/tasks)

- GET /tasks → Retrieve all tasks
- GET /tasks/{id} → Get a specific task
- POST /tasks → Create a new task
- PUT /tasks/{id} → Update a task
- DELETE /tasks/{id} → Delete a task

Subtasks API (/subtasks)

- GET /subtasks → Retrieve all subtasks
- GET /subtasks/task/{taskId} → Get subtasks for a specific task
- GET /subtasks/{id} → Get a specific subtask
- POST /subtasks → Create a new subtask
- PUT /subtasks/{id} → Update a subtask
- DELETE /subtasks/{id} → Delete a subtask

Users API (/users)

- GET /users → Retrieve all users
- GET /users/{id} → Get a specific user
- POST /users → Create a new user
- PUT /users/{id} → Update a user
- DELETE /users/{id} → Delete a user

We will implement a minimal set of these for this project to demonstrate the communications.

The starter repository contains the full source for both the frontend and backend, for this project all you need to do is build and execute the backend, build and connect to the Angular frontend using a browser, and take screenshots to show your work.

Overview of Required Steps

- Clone the repository
- Follow these steps to run the backend server:
 1. Use the psql command line tool to create the taskmanager_db (Horizon authentication is postgres/P@ssw0rd1).
 2. Install the JPA Buddy plug-in in IntelliJ.
 3. Configure the username/password in the application.properties file if you aren't using postgres/postgres.
 4. Build and run the server in IntelliJ.
 5. Use the "curls.bat" script in the TaskManager folder to add some data to the DB.
- After the server is running, move to the TaskManagerFrontEnd folder and execute the following commands to install TypeScript and its dependencies, install Angular, build the application, and start the development Web Server:

```
npm install
npm install -g @angular/cli
npm run build
npm start
```

- Connect to localhost:4200 and you should see the user, task, and subtask that was created by Curl on the backend. Take a screenshot and upload to the Canvas assignment.

Once you have completed these steps, start VS Code in the TaskManagerFrontEnd folder and familiarize yourself with the new code. Below are some key highlights:

Models Subfolder:

The models subfolder contains the primary domain objects, which correspond to the entity classes on the backend.

- user.ts (represents a user)
- task.ts (represents a task)
- subtask.ts (represents a subtask)

These files define the TypeScript interfaces or classes used to model the data structure shared between the frontend and backend.

Services Subfolder:

The services subfolder contains classes responsible for interacting with the backend server. These services mirror the backend's business logic layer. The classes import Angular's HttpClient library and provide CRUD operations (e.g., GET, POST, PUT, DELETE) to communicate with the backend's RESTful API.